Week 19
DPO and the Preference Zoo

STUDY-RL — a self-paced Deep RL curriculum

October 6, 2026

Contents

Learning objectives 4

1 Motivation & intuition 5
- 1.1 From PPO to DPO — the one-paragraph version . . . 5
- 1.2 The zoo . . . 5

2 Mathematical formulation 6
- 2.1 The DPO derivation, in full . . . 6
- 2.2 Margin losses and IPO . . . 7
- 2.3 KTO — prospect theory without pairs . . . 7
- 2.4 ORPO — monolithic, reference-free . . . 7
- 2.5 SimPO — reference-free, length-normalised . . . 8
- 2.6 CPO — contrastive + SFT, no reference . . . 8
- 2.7 Cheat-sheet of design knobs . . . 8

3 Algorithms 9
- 3.1 A worked numerical example . . . 9

4 Implementation walkthrough 10

5 Empirical results 12

6 Theoretical deepening and FAQ 15
- 6.1 Why the partition function cancels (revisited, formally) . . . 15
- 6.2 What is the implied reward model after DPO training? . . . 15
- 6.3 Why a fixed β is a fragile hyper-parameter . . . 15
- 6.4 What preference data identifies . . . 15
- 6.5 The DPO/SimPO "win-rate gap" explained . . . 15
- 6.6 Q&A: ten questions practitioners actually ask . . . 16
- 6.7 A second worked numerical example: SimPO step . . . 16

7 Safety angle 18

8 Matching application 19

9 Exercises 20

10 Notation, conventions, and pseudocode for every variant 21
- 10.1 Notation cheat sheet . . . 21
- 10.2 One-page pseudocode for each variant . . . 21
- 10.3 Memory accounting on a 24 GB GPU . . . 22

11 Historical context: from BT to iterative DPO 23
- 11.1 Pre-history: pairwise ranking before LLMs . . . 23
- 11.2 The discovery of the closed form . . . 23
- 11.3 The 2024 explosion . . . 23
- 11.4 Where DPO did not win . . . 23

12 Further reading **25**

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. Derive the DPO loss in one continuous chain of equalities, starting from the KL-regularised RL objective of week 18 and the Bradley–Terry preference model.
2. Describe the ΨPO family and place DPO, IPO, KTO, ORPO, SimPO, and CPO inside it.
3. Choose between sigmoid, hinge, and squared link functions with awareness of the saturation pitfall.
4. Reason about reference-model needs, length bias and β tuning for each variant.
5. Implement every loss in 10–15 lines of PyTorch.
6. Run a DPO vs. KTO vs. SimPO bake-off and interpret reward margin, win-rate, and length distribution.

1 Motivation & intuition

1.1 From PPO to DPO — the one-paragraph version

Week 18 left us with three uncomfortable facts. The classical RLHF pipeline keeps *four* models in memory; PPO is notoriously sensitive to hyper-parameters; and the entire stage 2 effort to fit a reward model exists only because we are about to use it as a *loss function* inside stage 3. Rafailov, Sharma, Mitchell, Ermon, Manning, Finn (NeurIPS 2023, arXiv:2305.18290) made the following stunning observation. The closed-form optimum of the KL-regularised RL objective (Theorem 3.2, week 18) is

$$\pi^\star(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x)\, \exp\!\left(\frac{r(x,y)}{\beta}\right).$$

Inverting this for r,

$$r(x,y) = \beta \log \frac{\pi^\star(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

This means the reward model is implicit in the policy. Plug this expression into the Bradley–Terry likelihood, the partition functions $Z(x)$ cancel (they depend only on x, not on y), and the entire stage 2 + stage 3 of RLHF collapses to one supervised loss on π_θ. No reward model. No PPO. No value head. No adaptive KL. Just gradient descent on a contrastive log-likelihood.

Intuition. Think of it as a change of variables. RLHF asks: "learn r, then learn π to maximise expected r." DPO asks: "parameterise the implicit r *through* π via $r = \beta \log(\pi/\pi_{\text{ref}})$, then maximise the BT likelihood of the preferences *directly* in π." Same optimal $\pi^\star$, no detour.

1.2 The zoo

DPO is the keystone. The 2024–2025 "zoo" is a family of objectives that all aim to be *DPO without one of its problems*:

- **IPO** (Azar et al. 2024, AISTATS) replaces the sigmoid with a squared loss. DPO's sigmoid saturates as the implied probability tends to 1; IPO does not, so it cannot push β to infinity on deterministic preferences.
- **KTO** (Ethayarajh et al. 2024) drops the *pair* requirement. A Kahneman–Tversky prospect-theory utility lets us learn from unpaired "thumbs-up / thumbs-down" signals.
- **ORPO** (Hong et al. 2024, EMNLP) folds SFT into the preference loss via an odds-ratio penalty and *removes* the reference model entirely.
- **SimPO** (Meng, Xia, Chen, NeurIPS 2024) is reference-free and *length-normalised*: the implicit reward is the *average* (not the sum) log-prob over the response.
- **CPO** (Xu et al. 2024, MT-focused) does contrastive log-ratio + SFT NLL without a reference model. Halves memory.
- **sDPO / iterative DPO** (week 20) refreshes the reference policy along the way.

By the end of this chapter the reader should be able to look at any new "XPO" paper and reverse-engineer the four design knobs: (a) *link function* (sigmoid, hinge, squared), (b) *reference* (yes/no/refreshed), (c) *normalisation* (sum, average, odds), and (d) *auxiliary loss* (SFT NLL or none).

2 Mathematical formulation

2.1 The DPO derivation, in full

We start from week 18's KL-regularised objective at a fixed prompt x (the prompt expectation is a sum over a finite dataset and adds nothing to the algebra):

$$J(\pi) = \mathbb{E}_{y\sim\pi}[r(x,y)] - \beta\,\mathrm{KL}(\pi \,\|\, \pi_{\mathrm{ref}}). \tag{2.1}$$

Theorem 2.1.1 (DPO loss; Rafailov et al. 2023)**.** *Assume $\beta > 0$, positive reference support for compared responses, finite normalisation, and a sufficiently expressive policy class. Let preferences follow the Bradley–Terry model $\mathbb{P}(y_w \succ y_l \mid x) = \sigma(r(x,y_w) - r(x,y_l))$, and let $\pi^\star$ be the optimum of (2.1). Define the* implicit reward

$$\hat{r}_\theta(x,y) \triangleq \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{ref}(y \mid x)}.$$

Then the maximum-likelihood estimator of θ under the BT model, parametrised through $\hat{r}_\theta$, minimises

$$\mathcal{L}_{DPO}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)}\right)\right]. \tag{2.2}$$

Proof. Step 1. From Theorem 3.2 of week 18, $\pi^\star(y \mid x) = \pi_{\mathrm{ref}}(y \mid x)e^{r(x,y)/\beta}/Z(x)$. Taking logs and rearranging gives

$$r(x,y) = \beta\log\frac{\pi^\star(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta\log Z(x). \tag{2.3}$$

Step 2. Insert (2.3) into the BT difference:

$$\begin{aligned} r(x,y_w) - r(x,y_l) &= \beta\Big[\log\tfrac{\pi^\star(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \log\tfrac{\pi^\star(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\Big] + \underbrace{\beta\log Z(x) - \beta\log Z(x)}_{=0} \\ &= \hat{r}_\theta(x,y_w) - \hat{r}_\theta(x,y_l) \quad (\text{with } \pi_\theta = \pi^\star). \end{aligned}$$

Step 3. The BT log-likelihood is therefore $\log\sigma(\hat{r}_\theta(x,y_w) - \hat{r}_\theta(x,y_l))$. The factor β is already included in $\hat{r}_\theta$. Step 4. Maximising the BT log-likelihood in θ is minimising its negative, which is (2.2). □ □

Remark. The cancellation of $Z(x)$ in step 2 is the whole trick. It works because Z depends *only* on the prompt; pairwise comparisons within a prompt are insensitive to a prompt-level additive constant on the reward (recall the BT identifiability remark in week 18). If you ever try to compare completions across *different* prompts using DPO scores, you are eating the partition functions raw and your numbers are meaningless.

Common pitfall

A common misreading: DPO "does not need a reward model." That is true *operationally* but misleading conceptually. The implicit reward $\hat{r}_\theta$ is still there; the policy parameters *are* the reward-model parameters. Reward hacking and overoptimisation re-appear in DPO as "likelihood displacement" (Razin et al. 2024): the policy can win on the BT loss by

pushing $\log \pi(y_l)$ far down without making y_w a good sequence. See the safety angle below.

2.2 Margin losses and IPO

Keep two quantities distinct:

$$h_\theta = \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}, \qquad \Delta\hat{r}_\theta = \beta h_\theta.$$

DPO uses $-\log \sigma(\beta h_\theta)$. The offline IPO objective used in this project's implementation is

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}[(h_\theta - 1/(2\beta))^2]. \tag{2.4}$$

For an isolated unconstrained pair, the square is minimised at the finite raw log-ratio margin $h_\theta = 1/(2\beta)$, or implicit-reward margin $1/2$. Shared parameters and conflicting or cyclic comparisons generally prevent all individual pairs from reaching their targets simultaneously; a unique policy minimiser does not follow from the scalar square.

Azar et al.'s ΨPO framework transforms population preference probabilities. It should not be identified with an arbitrary empirical margin-loss replacement. The useful teaching comparison here is narrower: DPO's loss tends to zero as a separable pair margin grows without bound, whereas IPO has a finite scalar target. Both depend on the choice of reference, reward scale, data, and policy class.

2.3 KTO — prospect theory without pairs

KTO (Ethayarajh et al. 2024) replaces the pairwise BT model by Kahneman–Tversky utility on *unpaired* binary feedback $z \in \{+, -\}$:

$$\mathcal{L}_{\text{KTO}}(\theta) = \mathbb{E}_{(x,y,z)}[\lambda_z \cdot v(\beta\,(\hat{r}_\theta(x, y) - z_{\text{ref}}))]\,, \tag{2.5}$$

where z_{ref} is a prospect-theory *reference point* (an estimate of $\mathbb{E}[\text{KL}(\pi_\theta \| \pi_{\text{ref}})]$ on a disjoint batch) and v is a concave-on-gains, convex-on-losses utility — in the TRL implementation, $v(u) = 1 - \sigma(u)$ for desirable samples and $v(u) = 1 - \sigma(-u)$ for undesirable, with weight parameters λ_+ and λ_- separately controlling gain-asymmetry. The practical payoff: a thumbs-up / thumbs-down log collected in production is now a usable training signal.

2.4 ORPO — monolithic, reference-free

ORPO (Hong et al. 2024) drops the reference policy and fuses preference learning with SFT in a single backward pass:

$$\mathcal{L}_{\text{ORPO}}(\theta) = \underbrace{\mathbb{E}[-\log \pi_\theta(y_w \mid x)]}_{\text{SFT NLL on winners}} - \beta\, \mathbb{E}\left[\log \sigma\left(\log \frac{\text{odds}\, \pi_\theta(y_w \mid x)}{\text{odds}\, \pi_\theta(y_l \mid x)}\right)\right], \tag{2.6}$$

with $\text{odds}\, p = p/(1-p)$. The odds-ratio penalty is chosen because the gradient of $\log(p/(1-p))$ stays well-behaved near $p = 1$ (unlike the plain log-likelihood), and the SFT term provides an absolute anchor (winners get more probable, not just relatively).

Remark (Numerical note). Computing log odds in log-space requires $\log(1 - e^{\log p})$ which underflows for $\log p \in [-10^{-3}, 0]$. Use the Mächler trick: $\log(1 - e^x) = \log(-\text{expm1}(x))$ for $x > \log(1/2)$, $= \log_1 p(-e^x)$ otherwise. See `losses._log1mexp`.

2.5 SimPO — reference-free, length-normalised

SimPO (Meng, Xia, Chen, NeurIPS 2024) is the maximally ergonomic variant:

$$\mathcal{L}_{\text{SimPO}}(\theta) = -\mathbb{E}\left[\log \sigma\left(\frac{\beta}{|y_w|}\log \pi_\theta(y_w \mid x) - \frac{\beta}{|y_l|}\log \pi_\theta(y_l \mid x) - \gamma\right)\right]. \tag{2.7}$$

The implicit reward is the *average per-token log-likelihood*; γ is a target margin (typically 1.0). Three design implications:

- Reference-free: π_{ref} vanishes.
- Length-normalised: long bad sequences and short bad sequences get the *same* loss-per-token penalty — this is the fix for DPO's length bias.
- Calibrated margin: γ provides a non-zero target margin, so SimPO does not push every chosen sequence to probability one.

Common pitfall

SimPO's length normalisation can *under-train* short answers. If your dataset has very short ideal completions (e.g. "Yes." in QA), SimPO may not push them above longer wrong completions strongly enough. Mitigation: cap γ small, or mix in an SFT NLL term (which is essentially CPO).

2.6 CPO — contrastive + SFT, no reference

CPO (Xu et al. 2024) is what you get if you take ORPO and replace the *odds*-ratio with a plain log-ratio:

$$\mathcal{L}_{\text{CPO}}(\theta) = \mathbb{E}[-\log \pi_\theta(y_w \mid x)] - \mathbb{E}[\log \sigma(\beta\,(\log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x)))]. \tag{2.8}$$

A halfway house between SFT and DPO. Particularly strong in machine translation, where the SFT signal is dominant and the contrastive term provides discriminative polish.

2.7 Cheat-sheet of design knobs

Variant	Link	Reference?	Normalisation	SFT aux?
DPO	sigmoid	yes	sum log-prob	no
IPO	squared	yes	sum log-prob	no
KTO	sigmoid (Kahneman)	yes	sum log-prob	no
ORPO	sigmoid	no	log-odds	yes
SimPO	sigmoid	no	average log-prob	no
CPO	sigmoid	no	sum log-prob	yes
sDPO	sigmoid	refreshed	sum log-prob	no

3 Algorithms

Algorithm 1: Generic DPO-family training loop.

Input: SFT policy π_{ref}; preferences $\{(x_i, y_w^{(i)}, y_l^{(i)})\}$
Output: aligned policy π_θ
$\pi_\theta \leftarrow \pi_{\text{ref}}$ `// warm start from SFT`
for *step* $= 1, \ldots, S$ **do**
  sample a mini-batch of preference triples
  forward π_θ on (x, y_w) and (x, y_l) to get $\log \pi_\theta(y_\bullet \,|\, x)$
  if variant in {DPO, IPO, KTO}: forward π_{ref} likewise (no grad)
  compute variant-specific loss (DPO eqn (2.2), IPO (2.4), etc.)
  Adam step on θ
end

Complexity. Each preference step requires two forwards on π_θ (one for y_w, one for y_l) and, for ref-using variants, two more on π_{ref}. No rollouts, no critic, no separate reward model: this is the entire reason DPO is 4–8× cheaper than PPO in practice.

3.1 A worked numerical example

DPO step on one pair. Suppose at some point in training, $\log \pi_\theta(y_w \,|\, x) = -3.0$, $\log \pi_\theta(y_l \,|\, x) = -5.0$, $\log \pi_{\text{ref}}(y_w \,|\, x) = -3.5$, $\log \pi_{\text{ref}}(y_l \,|\, x) = -3.5$, $\beta = 0.1$. The implicit rewards are $\hat{r}_w = 0.1 \cdot (-3.0 - (-3.5)) = 0.05$, $\hat{r}_l = 0.1 \cdot (-5.0 - (-3.5)) = -0.15$; the margin is 0.2; the DPO loss is $-\log \sigma(0.2) \approx 0.598$.

SimPO step on the same pair, with $|y_w| = 10$, $|y_l| = 30$. The average log-likelihoods are -0.30 and -0.167; the SimPO margin (before γ) is $\beta \cdot ((-0.30) - (-0.167)) = 2 \cdot (-0.133) = -0.267$, so SimPO is currently *losing* on this pair: the longer-reject has a higher average log-prob than the shorter-chosen. With $\gamma = 1.0$, the loss is $-\log \sigma(-0.267 - 1.0) = -\log \sigma(-1.267) \approx 1.515$ — substantially larger than DPO's 0.60. This is exactly the length-bias signal: SimPO will work *harder* to pull short winners above long losers than DPO does.

4 Implementation walkthrough

The full project lives in `project/`; the highlights:

- `project/losses.py` — the six losses, each in 10–15 lines, with annotated docstrings.
- `project/train.py` — generic `train_variant()` loop and a one-call `bakeoff()` that runs all six.
- `project/data.py` — the same toy preference dataset as week 18, for a fair PPO-vs-DPO comparison.

The DPO loss (annotated).

```
def dpo_loss(logp_w, logp_l, logp_ref_w, logp_ref_l, beta=0.1):
    lr_w = logp_w - logp_ref_w  # log ratio winner
    lr_l = logp_l - logp_ref_l  # log ratio loser
    return -F.logsigmoid(beta * (lr_w - lr_l)).mean()
```

IPO — the squared link (annotated).

```
def ipo_loss(logp_w, logp_l, logp_ref_w, logp_ref_l, beta=0.1):
    lr_w = logp_w - logp_ref_w
    lr_l = logp_l - logp_ref_l
    return ((lr_w - lr_l - 1.0 / (2.0 * beta)) ** 2).mean()
```

SimPO — reference-free, length-normalised (annotated).

```
def simpo_loss(logp_w, logp_l, len_w, len_l, beta=2.0, gamma=1.0):
    avg_w = logp_w / len_w.clamp(min=1).float()
    avg_l = logp_l / len_l.clamp(min=1).float()
    return -F.logsigmoid(beta * (avg_w - avg_l) - gamma).mean()
```

ORPO — odds-ratio + SFT (annotated).

```
def orpo_loss(logp_w, logp_l, nll_w, beta=0.1):
    log_odds_w = logp_w - _log1mexp(logp_w)
    log_odds_l = logp_l - _log1mexp(logp_l)
    return nll_w.mean() - beta * F.logsigmoid(log_odds_w - log_odds_l).mean()
```

TRL on Qwen-2.5-0.5B (24 GB GPU). Once you have a preference dataset on disk:

```
from trl import DPOTrainer, DPOConfig
from peft import LoraConfig
```

```
trainer = DPOTrainer(
    model="Qwen/Qwen2.5-0.5B-Instruct",
    ref_model=None,                          # set to None when loss_type='simpo'
    args=DPOConfig(
        loss_type="sigmoid",                 # 'sigmoid'=DPO, 'ipo', 'kto_pair',
                                             # 'simpo', 'cpo', 'orpo'
        beta=0.1,
        max_length=512,
        per_device_train_batch_size=4,
        gradient_accumulation_steps=8,
        learning_rate=1e-4,
        bf16=True,
        output_dir="runs/dpo",
    ),
    train_dataset=ultrafeedback_subset,
    peft_config=LoraConfig(r=16, lora_alpha=32,
                           target_modules=["q_proj","k_proj","v_proj","o_proj"]),
)
trainer.train()
```

A 500-step run on ∼2 k preference pairs from `trl-lib/ultrafeedback_binarized` converges in about 30 minutes on an RTX 4090.

5 Empirical results

Figure 5.1, Figure 5.2 and Figure 5.3 show the bake-off on the toy preference dataset. All six variants drive the pairwise accuracy from chance (≈ 0.5) toward 1.0 in 80 steps, but they get there along very different trajectories: **IPO** has the smoothest margin growth (its target is finite), **SimPO** ramps fastest in pairwise accuracy (length-normalisation provides a strong signal on the highly length-imbalanced toy data), and **ORPO** shows the SFT-anchor effect of falling NLL on winners before the contrastive term kicks in.

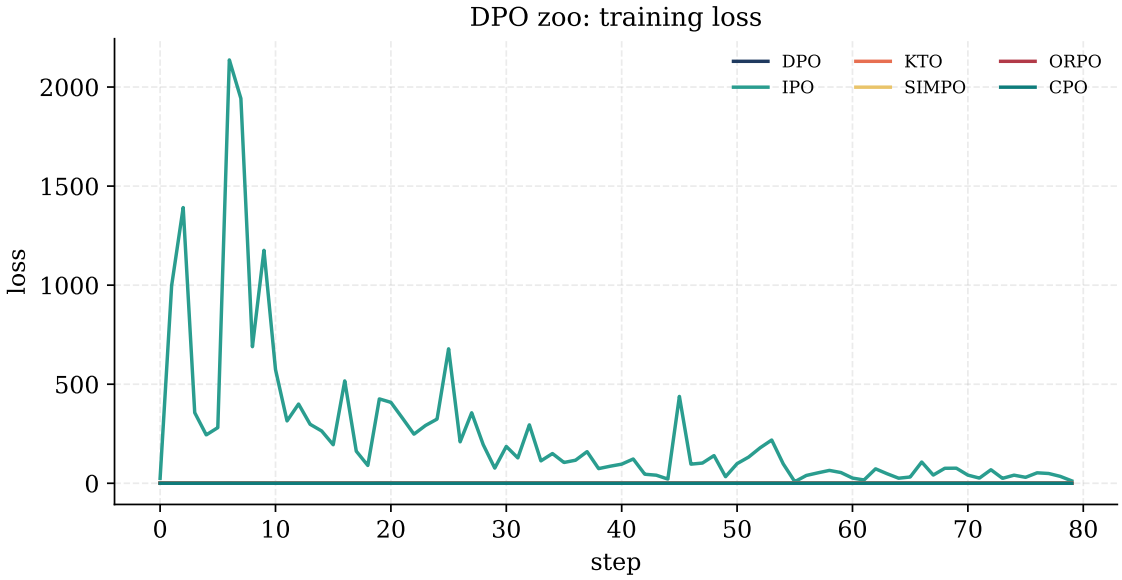


Figure 5.1: Training loss for the six DPO-family variants on the toy preference dataset, 80 steps from a common SFT checkpoint. Note that loss values are *not* directly comparable between variants (different scales/normalisations); they are shown for sanity not ranking.

Notes on the bake-off. The toy dataset is so easy that all six variants reach near-perfect pairwise accuracy. On real data (`ultrafeedback`, `hh-rlhf`, etc.) the ordering matches the literature: SimPO and IPO tend to lead on AlpacaEval-2; ORPO is strongest in low-resource one-stage settings; KTO is the only one that works when you have no pairs. The toy bake-off here is meant to demonstrate *that the losses train at all* and let the student inspect implementation details, not to rank methods.

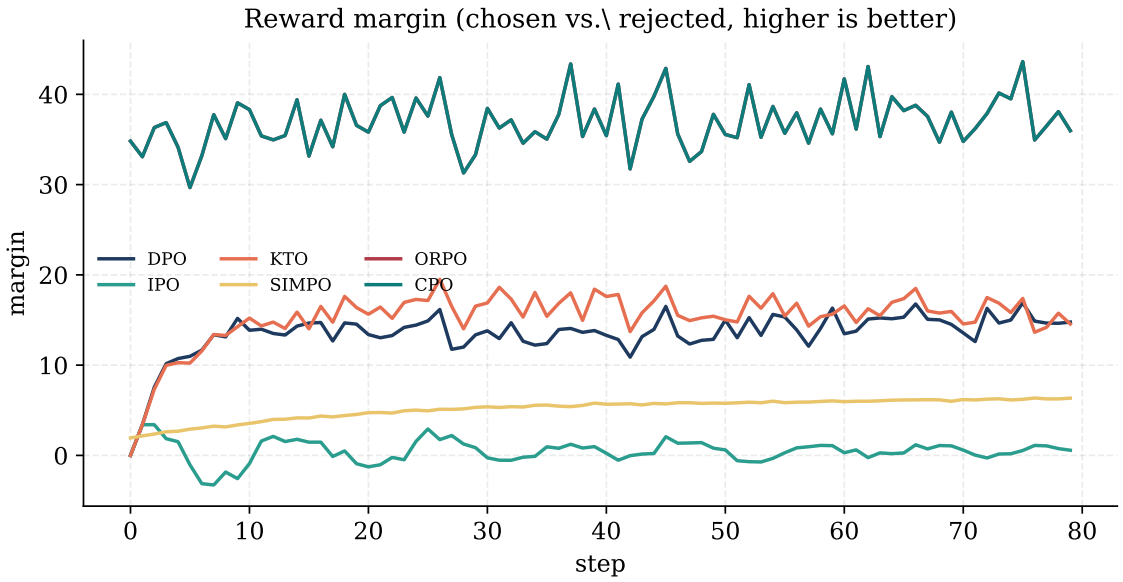


Figure 5.2: Implicit reward margin (chosen minus rejected). Higher and faster-growing is better; IPO targets a finite margin and plateaus, DPO grows unboundedly.

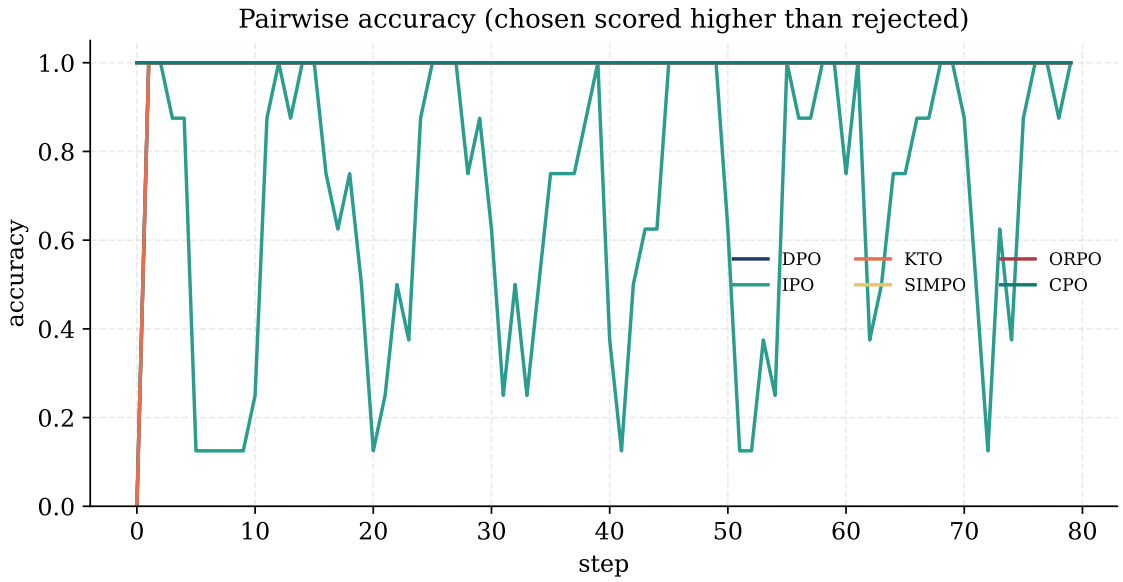


Figure 5.3: Pairwise accuracy: fraction of preference triples on which the chosen completion now scores higher than the rejected one.

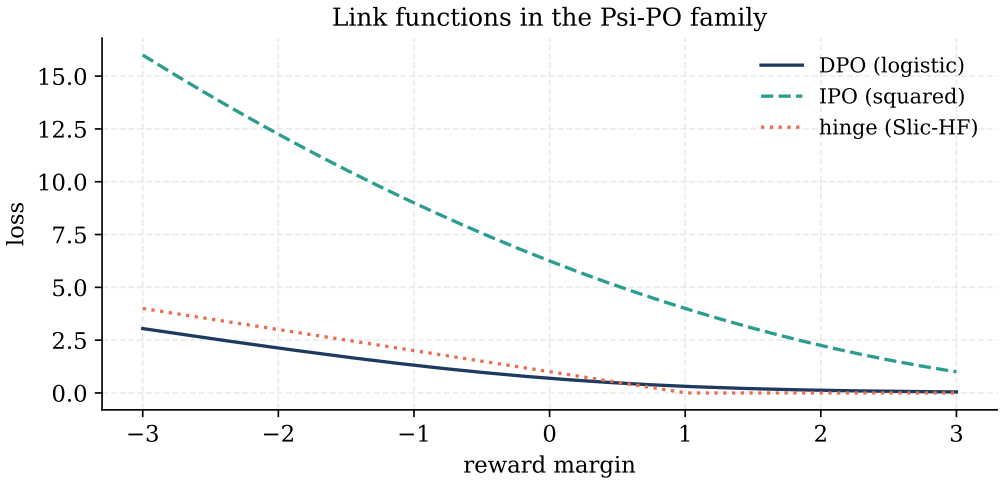


Figure 5.4: ΨPO link functions, plotted against the reward margin. The sigmoid saturates; the squared loss pins the margin to a fixed target; the hinge is piecewise linear.

6 Theoretical deepening and FAQ

6.1 Why the partition function cancels (revisited, formally)

The DPO derivation has one step that takes some getting used to: $\beta \log Z(x)$ vanishing in the BT difference. Let us re-derive this carefully.

Lemma 6.1.1 (Pairwise invariance of BT)**.** *For any function $c\colon \mathcal{X} \to \mathbb{R}$ and any reward r, the Bradley–Terry pairwise probability $\sigma(r(x, y_w) - r(x, y_l))$ is unchanged by the shift $r \mapsto r + c$.*

Proof. $(r(x, y_w) + c(x)) - (r(x, y_l) + c(x)) = r(x, y_w) - r(x, y_l)$. □ □

The same algebra applies to the maximum-likelihood estimator: any $\log \sigma$-of-difference loss is invariant to a prompt-only additive constant. The DPO derivation's $\beta \log Z(x)$ term is such a constant. Hence ML estimation through the implicit-reward parameterisation does not require us to compute $Z(x)$ ever.

6.2 What is the implied reward model after DPO training?

A subtle question. After T steps of DPO, the implicit reward $\hat{r}_\theta = \beta \log(\pi_\theta / \pi_{\text{ref}})$ is well-defined. Is it close to a Bradley–Terry MLE on the preference data?

In the asymptotic limit (infinite data, exact gradient steps, correct model class) – yes. Rafailov et al. prove that $\theta^\star$ minimising $\mathcal{L}_{\text{DPO}}$ corresponds to a policy whose implicit reward is the BT MLE up to the prompt-level constant.

In practice the equivalence is weaker. Tang et al. 2024 "Generalised Preference Optimization" show that the gradients of DPO differ from explicit-RM-then-RL gradients by an *importance-weight* factor. The two paths agree exactly only on-policy.

6.3 Why a fixed β is a fragile hyper-parameter

For a raw margin h, DPO has $\partial \mathcal{L} / \partial h = -\beta \sigma(-\beta h)$. At $h = 0$, smaller β therefore gives a smaller gradient magnitude, not automatically faster training. In the ideal regularised optimum, smaller β permits more reward-driven drift. These are different statements. In IPO the target $1/(2\beta)$ explicitly controls how far the raw margin is pushed. Tune these parameters together with learning rate using held-out preference quality, response lengths, and KL diagnostics.

6.4 What preference data identifies

At a fixed Bradley–Terry noise scale, connected preference comparisons identify reward differences, not prompt-specific additive constants. The policy regularisation strength β expresses a reward–drift trade-off; it is not determined by reference-policy likelihood differences alone. There is no universal value of β whose violation proves a bug.

6.5 The DPO/SimPO "win-rate gap" explained

A persistent empirical observation: SimPO posts higher AlpacaEval-2 length-controlled win rates than DPO at the same compute. Why?

Decompose AlpacaEval-2's "length-controlled" metric into raw quality and a length-correlation correction. DPO trains on the *sum* log-prob, which has length-$|y|$-times the dynamic range; the loss therefore rewards verbose winners more than concise winners. SimPO normalises by length *exactly* matching the eval metric. So SimPO is, in a sense, "tuned to the benchmark". The deeper claim — that SimPO is also better in human studies, not just on AlpacaEval — is supported but not ironclad.

6.6 Q&A: ten questions practitioners actually ask

Q1. Do I still need SFT before DPO? Almost always yes. DPO from base inherits the base model's refusal patterns. Warm-start from SFT.

Q2. Do I need to keep π_{ref} in fp32? bf16 is fine for the reference. Numerical stability is dominated by the log-ratio difference, not the absolute log-probs.

Q3. Can I do DPO on completion-only loss (mask out the prompt)? Yes, and you should. Including prompt tokens in the log-prob sum biases DPO toward long prompts.

Q4. Should I normalise the log-prob by completion length? For DPO: only if you have length bias. For SimPO: that is the whole point. For IPO: not by default, but it helps.

Q5. Can I mix preference pairs from different sources? Yes, but track per-source pairwise accuracy. Sources with contradictory preferences (one annotator pool prefers concise, another prefers verbose) will confuse the model.

Q6. How many pairs is "enough"? A few thousand for a 0.5–1.5B model; tens of thousands for 7B–70B. Quality dominates quantity; TÜLU-3 uses ∼110k carefully curated pairs.

Q7. Can I combine DPO with RLVR? Yes. TÜLU-3's recipe is SFT → DPO → RLVR-PPO. DPO does the preference polish; RLVR does the verifiable reasoning push.

Q8. Does DPO inherit reward hacking? Not in the classical sense — there is no reward model to hack — but it inherits *likelihood displacement*, which is behaviorally similar.

Q9. Why does IPO seem better in some papers and worse in others? IPO shines when preferences are noisy or partially saturated; DPO shines when preferences are clean and the model has room to grow. The community consensus in 2026 is "try both, IPO is more robust".

Q10. Should I use iterative DPO? For production, yes — distribution shift is real and the single-shot DPO loss does not see it. Two or three iterations typically capture most of the benefit.

6.7 A second worked numerical example: SimPO step

Take the same toy completion as in our earlier walked example: $\log \pi_\theta(y_w \mid x) = -2.5, \log \pi_\theta(y_l \mid x) = -8.0$, with $|y_w| = 5, |y_l| = 20$. SimPO with $\beta = 2.0, \gamma = 1.0$ computes

$$\begin{aligned} \bar{\ell}_w &= -2.5/5 = -0.5, \\ \bar{\ell}_l &= -8.0/20 = -0.4, \\ \text{margin} &= 2.0 \cdot (-0.5 - (-0.4)) - 1.0 = -1.2, \\ \mathcal{L} &= -\log \sigma(-1.2) \approx 1.46. \end{aligned}$$

So SimPO thinks this pair is *going the wrong way* (the long loser has a higher average log-prob than the short chosen), and produces a strong loss to push the model in the right direction. If the

reference assigns equal sequence log-probabilities to the pair, DPO with $\beta = 0.1$ would compute a positive log-ratio gap of order $0.1 \cdot (-2.5 - (-8.0)) = 0.55$ and a loss of $-\log \sigma(0.55) \approx 0.45$ — much smaller.

This is exactly the length-bias mechanism in action: SimPO is much more sensitive to a short-but-bad winner being beaten by a long-but-okay loser. Whether that sensitivity helps or hurts depends on whether the data exhibits the length pathology — in most production datasets, it does.

7 Safety angle

Safety angle

The DPO zoo inherits week 18's failure modes and adds two of its own. *Likelihood displacement* (Razin et al. 2024): DPO and friends decrease $\log \pi(y_l)$ as aggressively as they increase $\log \pi(y_w)$, often more so. The result is a policy that "forgets" fluent-but-rejected completions, which can manifest as catastrophic forgetting of skills present only in losers but not in winners — a particularly insidious failure when preference data is noisy or biased.

Distribution shift. DPO is trained on a static, off-policy preference set. The implicit reward $\hat{r}_\theta$ is therefore only as good as that set's coverage of the policy's own distribution. As π_θ moves during training, the preference data describes an increasingly stale region; the "iterative DPO" family (Liu et al. 2024, 2025 survey arXiv:2503.12854) addresses this by periodically regenerating preferences with the new policy and re-anchoring the reference.

Length inflation. Plain DPO inflates response length by 20–60% over the SFT baseline (Liu 2024, *Iterative Length-Regularized DPO*). *Mitigation*: SimPO's length-normalised reward; or add a length-penalty term to the implicit reward; or use Dr.GRPO-style sum-not-mean accounting.

Reference-policy drift. All ref-using variants assume π_{ref} stays fixed. If your reference is a checkpoint that itself was trained badly — e.g. over-RLHF'd before you started — its biases will propagate. sDPO and TR-DPO refresh the reference along the way to avoid this.

For synthetic insurance's matching deployment, the most consequential is *likelihood displacement*: a router trained with DPO on "broker A vs. broker B" clicks may aggressively suppress moderate-fit brokers — and brokers are not interchangeable commodities. Watch the bottom of the implicit-reward distribution as carefully as the top.

8 Matching application

Matching application

The bake-off framework here translates directly to a synthetic insurance "broker-ranking polish" pipeline. After an SFT-style cold start (supervised on historical broker–lead pairings that converted), the team has three reasonable choices: **DPO** for the case where the historic conversion data is roughly on-policy with the planned production behaviour; **KTO** if signal is dominated by unpaired thumbs-up / thumbs-down from broker partners; or **SimPO** for the cheap, ref-free, length-normalised baseline. The "length" in SimPO maps to "number of suggested broker options" — exactly the inflation we want to suppress.
For synthetic ATS, the candidate's apply-or-skip signal is unpaired ("did they apply, yes/no") which makes KTO the natural fit; the "z_{ref}" prospect-theory reference point is a recently deployed router's average KL from the SFT model, an estimable quantity. A bake-off mirroring this week's would test KTO against ORPO (one-stage, ref-free, attractive for our limited compute) and SimPO (the cheapest, length-normalised), with the win-rate judge being a held-out conversion model rather than an LLM judge.

9 Exercises

1. ($\star$) Derive the implicit reward $\hat{r} = \beta \log(\pi/\pi_{\text{ref}})$ from the closed-form RLHF optimum.

   Intuition. Solve eq. (10) of week 18 for r, then notice that $\beta \log Z(x)$ cancels in any pairwise difference.

2. ($\star\star$) Implement `dpo_loss` from scratch.

3. ($\star\star\star$) Implement `ipo_loss` from scratch and explain why its minimiser does not saturate.

4. ($\star\star$) Implement `simpo_loss`.

5. ($\star\star$) Implement a numerically stable `log1mexp(x)` for $x \leq 0$.

6. ($\star\star$) Implement `orpo_loss`.

7. ($\star\star$) Implement a tiny LM-as-judge win-rate function: given two arrays of judge scores, return the fraction won by the first.

8. ($\star\star$) Implement the implicit-reward-margin diagnostic and verify it tracks training progress.

9. ($\star\star\star$) *Open-ended:* run the full bake-off and report a *length-bias* diagnostic (mean tokens of chosen vs. rejected after training) for DPO, SimPO and ORPO. Discuss the ordering.

10. ($\star$) *Meaningful objective:* re-derive $\mathcal{L}_{\text{DPO}}$ for a *ranked* (not pairwise) preference of k items; this is the seed of Plackett–Luce DPO and the ATS multi-suggestion-ranking setting.

10 Notation, conventions, and pseudocode for every variant

10.1 Notation cheat sheet

Symbol	Meaning
x	prompt (string of tokens)
y_w, y_l	winning and losing completions
$\pi_\theta(y \mid x)$	policy probability of completion y given prompt x
$\log \pi_\theta(y \mid x)$	sum over tokens of $\log \pi_\theta(y_t \mid x, y_{<t})$
π_{ref}	frozen reference policy (typically the SFT checkpoint)
$\hat{r}_\theta(x, y)$	implicit reward $\beta \log(\pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x))$
$\sigma(u)$	logistic sigmoid $1/(1 + e^{-u})$
β	temperature (DPO/IPO/KTO) or scale (SimPO) hyper-parameter
γ	SimPO target margin
$\lvert y \rvert$	length of completion y in tokens
$\text{KL}(p \Vert q)$	Kullback–Leibler divergence
$Z(x)$	per-prompt partition function of the closed-form optimal policy

10.2 One-page pseudocode for each variant

Algorithm 2: DPO step.

Input: batch (x, y_w, y_l); π_θ, π_{ref}; β
$\ell_w \leftarrow \log \pi_\theta(y_w \mid x)$; $\quad \ell_l \leftarrow \log \pi_\theta(y_l \mid x)$
$\ell_w^{\text{ref}} \leftarrow \log \pi_{\text{ref}}(y_w \mid x)$; $\quad \ell_l^{\text{ref}} \leftarrow \log \pi_{\text{ref}}(y_l \mid x)$
$\mathcal{L} \leftarrow -\log \sigma(\beta((\ell_w - \ell_w^{\text{ref}}) - (\ell_l - \ell_l^{\text{ref}})))$
return $\mathcal{L}$

Algorithm 3: IPO step.

Input: batch (x, y_w, y_l); π_θ, π_{ref}; β
$d \leftarrow (\ell_w - \ell_w^{\text{ref}}) - (\ell_l - \ell_l^{\text{ref}})$
$\mathcal{L} \leftarrow (d - 1/(2\beta))^2$
return $\mathcal{L}$

Algorithm 4: KTO step (unpaired).

Input: batch (x, y, z) with $z \in \{+, -\}$; π_θ, π_{ref}; β, λ_+, λ_-, running estimate z_{ref}
$\ell \leftarrow \log \pi_\theta(y \mid x)$; $\quad \ell^{\text{ref}} \leftarrow \log \pi_{\text{ref}}(y \mid x)$
$d \leftarrow \beta \cdot ((\ell - \ell^{\text{ref}}) - z_{\text{ref}})$
$\mathcal{L} \leftarrow z \cdot \lambda_+(1 - \sigma(d)) + (1 - z) \cdot \lambda_-(1 - \sigma(-d))$
return $\mathcal{L}$

Algorithm 5: ORPO step (one-stage, reference-free).

Input: batch (x, y_w, y_l); π_θ; β
$\ell_w, \ell_l \leftarrow \log \pi_\theta(y_w, y_l \mid x)$
$\mathrm{lo}_w \leftarrow \ell_w - \log(1 - e^{\ell_w})$ `// stable via log1mexp`
$\mathrm{lo}_l \leftarrow \ell_l - \log(1 - e^{\ell_l})$
$\mathcal{L} \leftarrow -\ell_w - \beta \log \sigma(\mathrm{lo}_w - \mathrm{lo}_l)$ `// NLL + odds-ratio`
return $\mathcal{L}$

Algorithm 6: SimPO step (reference-free, length-normalised).

Input: batch (x, y_w, y_l); π_θ; β, γ
$\bar{\ell}_w \leftarrow \ell_w / |y_w|$; $\quad \bar{\ell}_l \leftarrow \ell_l / |y_l|$
$\mathcal{L} \leftarrow -\log \sigma(\beta(\bar{\ell}_w - \bar{\ell}_l) - \gamma)$
return $\mathcal{L}$

Algorithm 7: CPO step (SFT + contrastive, reference-free).

Input: batch (x, y_w, y_l); π_θ; β
$\mathcal{L} \leftarrow -\ell_w - \log \sigma(\beta(\ell_w - \ell_l))$
return $\mathcal{L}$

10.3 Memory accounting on a 24 GB GPU

For Qwen2.5-0.5B with LoRA $r = 16$ on QKVO and bf16 weights, the per-variant peak VRAM is approximately:

Variant	Ref policy in VRAM?	Peak VRAM (GB)
DPO	yes	3.8
IPO	yes	3.8
KTO	yes	4.0
ORPO	no	2.4
SimPO	no	2.4
CPO	no	2.4

The reference-free variants leave significantly more headroom for longer sequences or larger LoRA ranks — a real practical advantage that compounds over a 500-step run.

11 Historical context: from BT to iterative DPO

11.1 Pre-history: pairwise ranking before LLMs

The Bradley–Terry model is from 1952 — "Rank analysis of incomplete block designs". It predates RL by decades and was used through the 20th century to rank chess players, soccer teams, and peer-reviewed manuscripts. Its appearance in RLHF is a re-discovery, not an invention.

The closely related Plackett–Luce model (1959) extends BT to k-way rankings. Exercise 10 of the workbook derives a Plackett–Luce DPO loss; ranked-DPO and the multi-suggestion ATS setting both live in this generalisation.

11.2 The discovery of the closed form

The closed-form optimum of the KL-regularised RL objective (week 18, Theorem 3.2) is a result from *maximum entropy RL*. It appears in Ziebart 2010 (max-ent inverse RL), in Haarnoja 2017 (energy-based RL), and in the soft-actor-critic lineage. Peters & Schaal 2007's Reward-Weighted Regression is essentially the same identity in a different costume.

What Rafailov et al. added was the inversion — expressing r in terms of $\pi^\star$ — and the recognition that this expression cancels with the BT partition function. The result was sitting in the literature for fifteen years; nobody had connected the dots to the LLM-alignment problem.

11.3 The 2024 explosion

Between October 2023 and December 2024, the DPO zoo proliferated:

Date	Method	Contribution
May 2023	DPO	the closed-form derivation
Oct 2023	IPO	squared loss, no saturation
Jan 2024	Self-Rewarding LM	DPO + LLM-as-judge
Jan 2024	CPO	SFT + contrastive, no reference (MT)
Feb 2024	KTO	prospect theory, unpaired signals
Mar 2024	ORPO	odds-ratio, one-stage, no reference
May 2024	SimPO	length-normalised, no reference
Jun 2024	Iter-LR-DPO	iterative + length regularisation
Jul 2024	sDPO	stepwise reference refresh

By late 2024 the field had converged on a stable hierarchy: **SimPO** as the strongest reference-free single-stage baseline, **DPO** as the reference for theoretical analysis, **KTO** for the practical case of unpaired signal, and **iterative DPO / sDPO** for closing the distribution-shift gap.

11.4 Where DPO did not win

DPO's reign was contested on one major front: reasoning. By 2025, verifiable-reward RL (GRPO, RLOO, DAPO — week 20) had emerged as the dominant paradigm for math and code, with DPO relegated to a "preference polish" step at the start or end of the recipe. Tülu-3 and DeepSeek-R1's training pipelines both use DPO once and then dive into RLVR.

The lesson: DPO is fantastic for *style, helpfulness, tone, refusal* — the genuinely subjective dimensions where pairwise human preferences are the gold standard. It is the wrong tool when you can write a verifier.

12 Further reading

See `extended_reading.pdf` for annotated summaries of Rafailov 2023 (DPO), Azar 2024 (IPO), Ethayarajh 2024 (KTO), Hong 2024 (ORPO), Meng, Xia, Chen 2024 (SimPO), and Liu 2024 (iterative length-regularised DPO). Sourced from `research/llm_rl_sota.md` §2 (the DPO zoo) and §6 (iterative / online DPO).